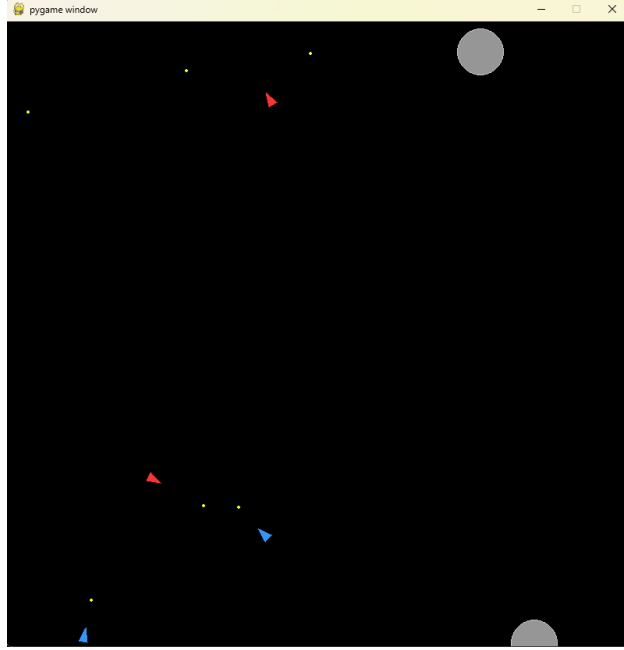


Asteroids Inspired Multi-Agent Reinforcement Learning Battle Simulator

Skye Crocker*

University of Colorado, Boulder, Colorado, 80305



Abstract- Agents are trained using Proximal Policy Optimization, a Deep Reinforcement Learning algorithm, in a custom game environment inspired by the Atari game Asteroids. Using similar controls and physics, the agents must work as a team to duel against an opposing team of agents while also avoiding asteroids. The agent is evaluated against passive opponents in a pursuit style game, head to head combat against aggressive opponents, and in multi-agent scenarios. The agent outperforms an expert designed heuristic bot in the most difficult pursuit scenario while achieving slightly worse, though nearly similar, performance in combat.

I. Introduction

Video games have historically been the gold standard for testing and benchmarking artificial intelligence (AI) and reinforcement learning (RL) algorithms. Atari games in particular have been a prominent benchmark and testing ground since Deep Q Learning (DQN) was first used to achieve human level performance back in 2015. [1] However, these games involve only a single intelligent agent acting in an environment. Many real world applications of reinforcement learning require several agents to work together to achieve a desired outcome. Some examples include autonomous cars, robots in a warehouse, search and rescue drone swarms, and space surveillance sensor networks. As if single agent RL wasn't already difficult enough, Multi-Agent Reinforcement Learning (MRL) is notoriously difficult due to challenges such as non-stationarity of environments, the credit assignment problem, competing notions of optimality and equilibria, and the curse of dimensionality as the number of agents increases. [2] The goal of this project was to create a simple multi-agent Atari-like environment, inspired by the game Asteroids, and use it to get started with MRL. In particular, the environment is so lightweight that it can be used to rapidly prototype and incorporate creative mechanics and scenarios to foster teamwork, various missions, and in a fun and exciting, human playable and verifiable

*M.S. Robotics Candidate, College of Engineering and Applied Sciences, 1111 Engineering Drive 422 UCB Boulder, CO 80309-0422

way. For our algorithm implementation, the minimum goal was to train a single agent to defeat an easy, non-threatening heuristic bot, and the primary goal was to achieve intelligent performance in a 2v2 team battle against a more capable expert designed heuristic policy. See the project design and scope disclaimer section VII for info on how this changed from the original proposal.

II. Background and Related Work

A. Environment Creation

Creating reinforcement learning environments today has never been easier thanks to APIs like OpenAI Gym [3] and PettingZoo [4]. OpenAI Gym is a framework that forces a consistent programmatic pattern (obs, reward, terminated, truncated, info = env.step(action)) for easy interface with RL agents. PettingZoo is the multi-agent counterpart to Gym which allows providing observations and rewards to multiple agents at each step, supports turn based or parallel actions and tracks activated vs dead agents. Furthermore, packages such as supersuit [5], a wrapper for PettingZoo, provide additional utilities and pre-processing conveniences such as vectorizing the environment or handling the observations and neural network inputs for dead agents.

B. Proximal Policy Optimization

Proximal Policy Optimization (PPO) is a more recent state of the art algorithm introduced in 2017 which showed improved performance over DQN, Policy Gradient, and Trust Region methods on Atari benchmarks. [6] At a high level, PPO works by collecting a batch of experience from the environment, using that data to update the policy, and then repeating. It combines ideas from Actor-Advantage-Critic (A2C) (having multiple workers) and Trust Region Policy Optimization (TRPO) (it uses a trust region to improve the actor). The key idea is that each update should not change the policy too drastically. If the new policy strays too far from the old one, performance can collapse. PPO handles this by clipping the update, essentially putting a limit on how much the policy is allowed to change in a single step.

This is captured by the clipped surrogate objective:

$$L(s, a, \theta_k, \theta) = \min \left(\frac{\pi_\theta(a|s)}{\pi_{\theta_k}(a|s)} A^{\pi_{\theta_k}(s, a)}, \text{clip} \left(\frac{\pi_\theta(a|s)}{\pi_{\theta_k}(a|s)}, 1 - \epsilon, 1 + \epsilon \right) A^{\pi_{\theta_k}(s, a)} \right) \quad (1)$$

where the ratio $\frac{\pi_\theta(a|s)}{\pi_{\theta_k}(a|s)}$ measures how much the policy has changed, A is the advantage estimate (how much better an action was compared to what was expected), and ϵ is a small hyperparameter that caps how far the ratio can deviate from 1. A value function loss and entropy bonus are then added to encourage accurate value estimates and continued exploration. This and additional equations are taken from the OpenAI Spinning Up documentation [7]. Please check out the website for a more in depth explanation of PPO. Our implementation of PPO comes from a library of reinforcement learning algorithms, stable-baselines3. [8]

C. Training Techniques

Oftentimes what is more important than the algorithm itself is training techniques such as reward shaping and curriculum learning. Generally, 2 player 0-sum games only have rewards of -1, 0, or +1 for a loss, draw, and win respectively. These sparse rewards can make it very difficult for the agent to find any signal to figure out what it is doing right or wrong. Reward shaping [9] is the process of injecting additional rewards, usually micro-rewards, which provide a small but dense signal to encourage or discourage behaviors and states. However, one must be careful of reward hacking, in which the agent foregoes the original desired goal to maximize its reward through undesired means.

Curriculum learning [10] follows a similar motivation, in that oftentimes dropping an agent into the chaos of the full environment can be too complex to figure out what is working or isn't working, thus it is first placed in a simple environment with a basic and easy task which it should be able to master. Once that task is mastered, the complexity of the environment is increased a little to give it a new challenge, not dissimilar to the way humans learn and progress. This can be especially important for PPO when compute time is limited, as PPO is highly sample inefficient and generally performs better when it can run for a very long time [6], which is not always an option.

III. Problem Formulation

The space battle environment is formulated as a Markov Game (or stochastic game) defined by the tuple $(\mathcal{I}, S, \{A^{(i)}\}_{i \in \mathcal{I}}, T, \{R^{(i)}\}_{i \in \mathcal{I}}, \gamma)$, where $\mathcal{I}$ represents the set of all active agents and $\gamma = .99$ is the discount factor. For neural network input consistency, agents technically only receive an observation of the k nearest entities of each type, so technically in theory this is actually a partially observable markov game, but for all scenarios in this project the number of entities in the game was strictly less than k , thus for the purposes of this report it is a fully observable markov game.

A. State Space (S)

The global state $s_t \in S$ contains the complete physical information of all entities in the environment at time t , defined as the union of all active ships, asteroids, and bullets:

$$s_t = (\mathcal{E}_{ships}, \mathcal{E}_{asteroids}, \mathcal{E}_{bullets})$$

Every entity $e \in s_t$ is characterized by a 2D position vector $p = (x, y)$ and a velocity vector $v = (\dot{x}, \dot{y})$ and a radius size r . Additionally, all ships have an angle θ as well as a shoot cooldown value, and bullets have a lifespan value.

B. Joint Action Space (A)

At each timestep, every active agent $i \in \mathcal{I}$ independently selects an action $a_t^{(i)} \in A^{(i)}$.

$$A^{(i)} = A_{thrust} \times A_{turn} \times A_{shoot}$$

Where A_{thrust} can take on a value of 0 (no thrust) or 1 (thrust), A_{turn} can take on a value of 0 (no turn), 1 (turn left) or 2 (turn right), and A_{shoot} can take a value of 0 (don't shoot) or 1 (do shoot), for a total of $2 \times 3 \times 2 = 12$ possible actions. These independent actions form the joint action vector at time t :

$$\mathbf{a}_t = (a_t^{(1)}, a_t^{(2)}, \dots, a_t^{(n)})$$

C. Transition Function (T)

The state transition function $T(s_{t+1}|s_t, \mathbf{a}_t)$ is implicitly defined by the environment's discrete-time physics simulator.

Rotation and Thrust Mechanics The heading angle θ of a ship is updated based on the discrete turning action and the fixed angular velocity ω :

$$\theta_{t+1} = \theta_t + a_{turn} \cdot \omega \quad (2)$$

Following the angular update, the new velocity vector is calculated by adding the applied thrust force F_{thrust} resolved along the ship's new heading, scaled by the environmental drag coefficient C_{drag} (representing velocity retention per frame):

$$\begin{bmatrix} \dot{x}_{t+1} \\ \dot{y}_{t+1} \end{bmatrix} = \left(\begin{bmatrix} \dot{x}_t \\ \dot{y}_t \end{bmatrix} + a_{thrust} \cdot F_{thrust} \begin{bmatrix} \cos \theta_{t+1} \\ \sin \theta_{t+1} \end{bmatrix} \right) C_{drag} \quad (3)$$

Positional Translation and Toroidal Wrapping The positional vectors of all active entities are updated using implicit Euler integration. To simulate the unbounded nature of space, coordinates are mapped to a toroidal topology using modulo arithmetic bounded by the environment dimensions W and H :

$$x_{t+1} = (x_t + \dot{x}_{t+1}) \bmod W \quad (4)$$

$$y_{t+1} = (y_t + \dot{y}_{t+1}) \bmod H \quad (5)$$

Projectile Initialization When an agent executes a shoot action ($a_{shoot} = 1$) and satisfies the weapon cooldown constraint ($c_t \leq 0$), a new bullet is spawned. Its initial position $\mathbf{p}_b$ is offset from the ship's center by the ship's radius R_{ship} plus a safe margin δ :

$$\mathbf{p}_b = \mathbf{p}_{ship} + (R_{ship} + \delta) \begin{bmatrix} \cos \theta_{t+1} \\ \sin \theta_{t+1} \end{bmatrix} \quad (6)$$

The bullet's initial velocity $\mathbf{v}_b$ inherits the ship's current velocity plus the intrinsic muzzle velocity V_{bullet} :

$$\mathbf{v}_b = \mathbf{v}_{ship} + V_{bullet} \begin{bmatrix} \cos \theta_{t+1} \\ \sin \theta_{t+1} \end{bmatrix} \quad (7)$$

Collision Dynamics The shortest distance d between any two entities i and j on the wrap-around topology is calculated using wrapped delta components:

$$\Delta x = \min(|x_i - x_j|, W - |x_i - x_j|) \quad (8)$$

$$\Delta y = \min(|y_i - y_j|, H - |y_i - y_j|) \quad (9)$$

$$d(i, j) = \sqrt{\Delta x^2 + \Delta y^2} \quad (10)$$

A fatal collision $C(i, j)$ is registered if this distance is strictly less than the sum of their physical hitboxes $R_i + R_j$:

$$C(i, j) = \begin{cases} 1 & \text{if } d(i, j) < R_i + R_j \\ 0 & \text{otherwise} \end{cases} \quad (11)$$

D. Reward Function (R)

The base environment provides only a reward at the end of each game. The game ends when only agents from one of the teams remains. The rewards are a -1 for a loss, 0 for a draw (timeout or mutual collision) and 1 for a win. If a game does not end after 1000 frames, it is considered a timeout draw. See the Solution Approach IV for a description of the reward shaping wrapper used for training.

IV. Solution Approach

A. Proximal Policy Optimization

The algorithm used to solve this problem was PPO. This was taken directly from stable-baselines3 with *learning_rate* = $3e-4$, *batch_size* = 256, *entropy_coefficient* = 0.1, and MLPPolicy. The input to the neural network is a 109 vector with the normalized velocity, heading and fire cooldown of itself (5 dim), and the normalized relative positions (2 dim), velocity (2 dim), and a one hot encoding of the entity type (4 dim - asteroid, bullet, opponent, ally) (8 dim total) for the nearest 5 asteroids, 3 enemies, 2 allies, and 3 bullets.

A technical description of PPO can be found in in the background section II.B, but the following pseudocode from [7] also provides an explanation for how the algorithm works.

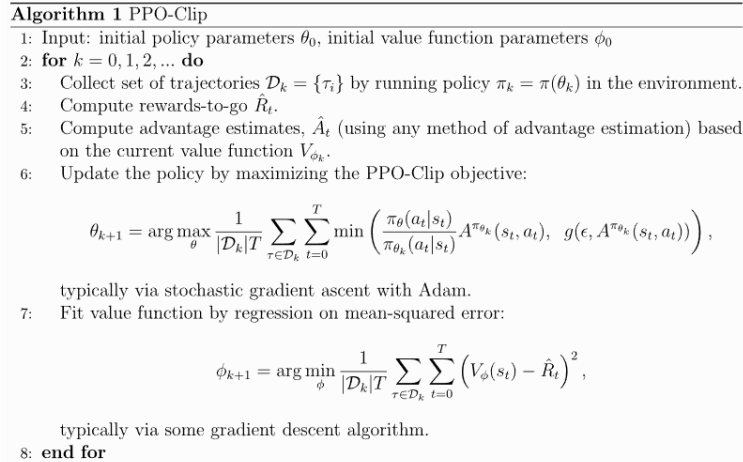


Fig. 1 Algorithm Source: Spinning up with OpenAI [7]

B. Reward Shaping

To overcome sparse terminal rewards, a reward shaping wrapper is applied to add rewards across four gameplay components. Game-ending events give a scaled payout $100 \cdot R_{base}$, and a constant -0.05 time penalty discourages stalling. Moving towards the opponent is encouraged through a heading alignment reward: $0.02(1 - |\Delta\theta|/\pi)$ where $\Delta\theta$ is the error in heading alignment to the nearest enemy, and a proximity gradient $0.02(1 - d_{enemy}/d_{max}) - 0.01$ rewarding closing distance, where d_{enemy} is the distance to the enemy and d_{max} is the maximum possible map distance. When the agent fires, it receives an accuracy reward $\max(0.05(1 - |\Delta\theta|/0.5), -0.01)$ that turns negative for angular errors exceeding 0.5 rad, discouraging blind fire. Finally, linear repulsor fields penalize the agent for getting close to nearby hazards, contributing $0.50(1 - d_h/W_h)$ where d_h is the distance to hazard h and W_h is the radius where the penalty zone begins, 150 for asteroids and 100 for bullets, summed over all nearby hazards $\mathcal{H}$. The total reward is:

$$r_t^{(i)} = R_{terminal} + R_{time} + R_{aim} + R_{prox} + R_{shoot} + R_{hazard}$$

C. Curriculum Learning

To foster a healthy learning environment, the agent was trained in 3 sequential scenarios of increasing complexity and difficulty. (Many different environments were tried and agents trained on, but the final one shown in the results section used only these 3).

1. Level 1: Basic Target Practice

In the first scenario in the curriculum, a fresh neural network is trained from scratch in the simplest possible environment. 1 blue agent, 1 red agent, no asteroids. The opposing red agent is a dummy which does not perform any actions and remains stationary. The agent is trained to target and destroy the opponent for 2 million steps.

2. Level 2: Moving Target and Hazards

In the next scenario, the weights learned from the previous scenario are put into a new environment. Once again, this is a 1v1 scenario, but now there are two indestructible asteroids which both agents must avoid. The red agent is once again a heuristic dummy, but now it uses a heuristic to turn and thrust away from asteroids as they approach, so it will not be killed by the hazards and requires the agent to shoot it. Notably, the dummy agent does not register bullets as hazards, so it is not too difficult to target. However, when no hazard is close, the heuristic bot does randomly turn and/or thrust with $p=0.1$ each frame providing minor constant movement, forcing the agent to hit a moving target. The agent is trained in this environment for an additional 2 million steps.

3. Level 3: Aggressive Moving Target and Hazards

This scenario is similar to the previous in terms of number of agents and asteroids. But now the bot opponent also flees from bullets, and shoots randomly. The agent must learn to hit a target that is actively trying to avoid being hit, while also avoiding stray enemy bullets. The agent is trained in this environment for 8 million steps.

4. (Bonus) Level 4: 2v2 vs Expert Opponents

In this scenario, there are now two agents on each team, and the red team is comprised of expert designed heuristic agents that actively beeline and shoot at the agent, while maintaining the evasion logic of the previous scenario bots. The agent was trained in this setting for another 8 million steps. However, the result of this process resulted in an agent that performed worse in all benchmarks, including the one it was trained in, so it is not used in the final evaluation.

V. Results

After training on the first 3 scenarios above, the resulting agent is tested against various heuristic bots in various scenarios, categorized into pursuit performance and combat performance.

A. Pursuit Performance

The pursuit scenarios test the agents ability to track down and destroy a non-threatening opponent. The agent is tested in 3 different environments corresponding to the training phases:

- **Level 1:** 1v1 in Open Space (no asteroids)
- **Level 2:** 1v1 with 2 asteroids
- **Level 3:** 2v2 with 2 asteroids

For each environment, the pursuit agent is also pitted against 3 variations of the evasion agent, once again corresponding to the training phases:

- **Static Agent:** A stationary dummy agent which does nothing.
- **Evasive (Easy):** The agent thrusts away from asteroids, and otherwise makes random tame movements
- **Evasive (Advanced):** The agent thrusts away from asteroids, bullets, and the enemy players, and otherwise makes random tame movements. It also shoots randomly to add additional hazards.

Additionally, to compare the performance against a baseline, the heuristic combat agent is also evaluated in these same scenarios at 4 levels. The combat agent maintains the same evasion logic as the evasion bots, and it prioritizes evasion. But when not near a hazard, it will turn and fly towards the nearest opponent and shoot at them. 4 levels of this agent are defined by how often it takes the expert heuristic "optimal" action. They all use the same logic but with probability .25/.5/.75/1. will actually take that action, and will otherwise perform a random action each frame.

Every combination of scenario was ran 1000 times. If the pursuit agent died or did not defeat the opponent within 1000 frames it failed. The results can be seen in Fig 2. Notably, the PPO agent outperforms the heuristic policy against the most difficult evaders, especially in the open space environment, but worse against the easier targets.

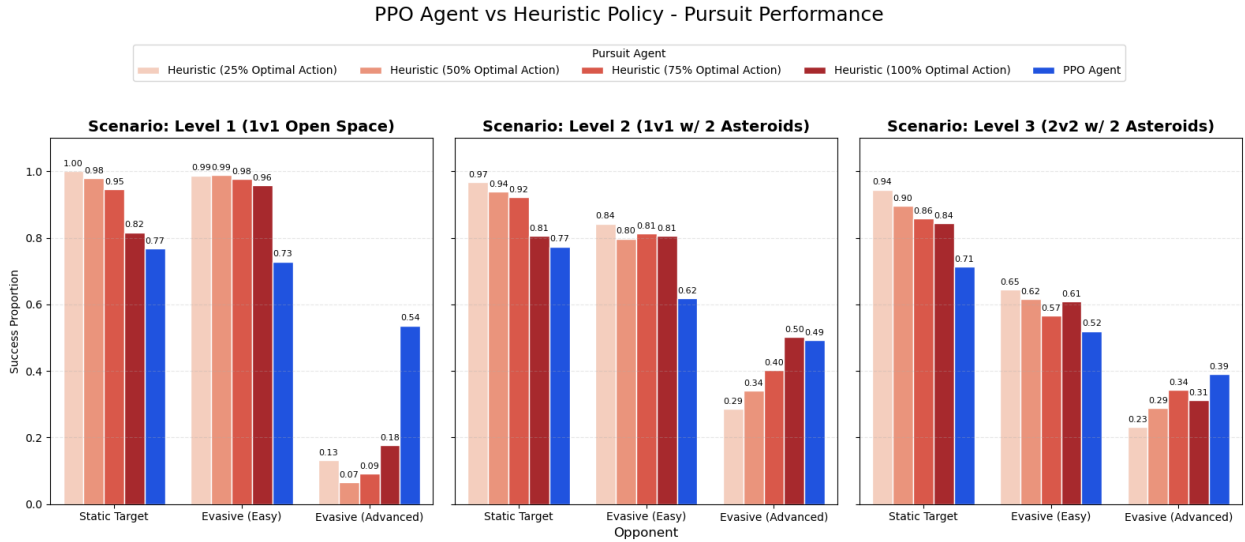


Fig. 2 Evaluation of agent's pursuit performance. The agents are evaluated across 3 environment scenarios and 3 target opponents for 1000 trials each.

B. Combat Performance

In the combat performance section, we use the same 3 environment setups as before, but now the PPO agent is directly pitted against the expert designed heuristic agents in a kill or be killed fight.

Once again, the heuristic agents are scaled at 4 levels of difficulty corresponding to the probability that they take the optimal action, and each matchup is ran 1000 times. A draw occurs when time runs out and both teams are still alive, or both die at the same time due to collision. The results can be seen in Fig 3. Overall, the PPO agent outperforms all the handicapped heuristic policies, but is bested by the 100% optimal heuristic policy, though it puts up a good fight.

VI. Discussion and Conclusions

Overall, the trained agent saw limited, though very real and promising success. While the heuristic agents struggle to pin down a moving evasive target, especially in open space, the PPO agent is able to predict and outmaneuver the advanced evader with much greater success. The fact this type of agent was the most recent one trained on may explain why the PPO agent performs worse on the easier targets.

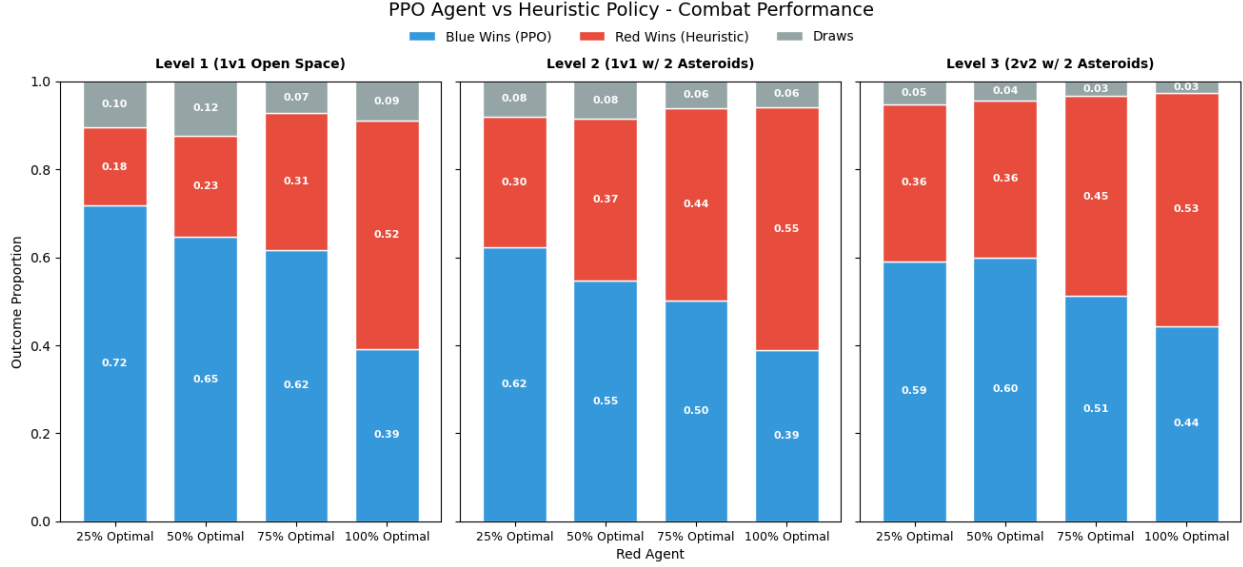


Fig. 3 Evaluation of agent’s combat performance. The PPO agent is evaluated across 3 environment scenarios and 4 combat opponents for 1000 trials each. Blue shows the win proportion, red the lose proportion, and gray draw.

In a dog fight, the PPO agent is able to achieve superiority over all heuristic agents that occasionally blunder and perform a random action instead of the heuristic-optimal action. While the fully optimal heuristic policy outperforms the PPO agent, it still puts up a good fight.

Notably, the agent is able to achieve this performance having never fought a combat opponent. Its entire training was based on a pursuit environment. Furthermore, while the network was designed to handle training with multiple intelligent agents at once, all the training of the evaluated bot was done in a solo 1v1 against a heuristic opponent (For this reason you could argue the problem formulation was just an MDP rather than a markov game). Despite this it was still able to generalize to multi-agent environments.

There are many potential improvements which the author is excited about:

- The agent could very likely be improved significantly simply by training for much longer.
- The agent has yet to train (successfully) in a combat setting.
- This project focused only on single agent RL, but the hope is that it will provide a good environment for MARL.
- Self vs self play has yet to be attempted, which may show exciting results with the right conditions.
- The environment can easily be expanded for heterogeneous agent systems, as well as scenarios that explicitly require teamwork and coordination for future fun and research.
- Ideally the state can be put into javascript for rendering a beautiful visualization and web application.

In conclusion, the project was a success based on our minimum and primary levels goals of learning to defeat a dummy agent, and the ability to show adequate/intelligent performance in a multi-agent setting against heuristic opponents. The stretch goal was not reached.

VII. Disclaimer: Project Design and Scope

The original project proposal focused on an asymmetric partially observable stochastic game in which sensor agents try to keep track of maneuvering satellite agents. The problem was reformulated into a fully observable markov game to make things a little easier for a first time RL project. While the physics and environment of the game changed, the levels of success were largely untouched and easily adapted. For example, minimum success is still a 1v1 against a dummy opponent, and main approach is still a small team vs team environment against a stronger heuristic opponent.

VIII. Contributions and Release

All work in this project was done by Skye Crocker, the sole author. The author grants permission for this report to be posted publicly.

Algorithms Note: The heuristic policy was implemented by myself. The PPO algorithm was implemented off the shelf from stable-baselines3

References

- [1] Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., et al., “Human-level control through deep reinforcement learning,” *nature*, Vol. 518, No. 7540, 2015, pp. 529–533.
- [2] Albrecht, S. V., Christianos, F., and Schäfer, L., *Multi-Agent Reinforcement Learning: Foundations and Modern Approaches*, MIT Press, 2024. URL <https://www.mar1-book.com>.
- [3] Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J., and Zaremba, W., “OpenAI Gym,” *arXiv preprint arXiv:1606.01540*, 2016.
- [4] Terry, J. K., Black, B., Grammel, N., Jayakumar, M., Hari, A., Sullivan, R., Santos, L., Perez, R., Horsch, C., Dieffendahl, C., Williams, N. L., Lokesh, Y., and Ravi, P., “PettingZoo: Gym for Multi-Agent Reinforcement Learning,” *arXiv*, 2020. <https://doi.org/10.48550/arxiv.2009.14471>.
- [5] Terry, J. K., Black, B., and Hari, A., “SuperSuit: Simple Microwrappers for Reinforcement Learning Environments,” *arXiv preprint arXiv:2008.08932*, 2020.
- [6] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O., “Proximal Policy Optimization Algorithms,” *arXiv preprint arXiv:1707.06347*, 2017. <https://doi.org/10.48550/arxiv.1707.06347>.
- [7] Achiam, J., “Spinning Up in Deep Reinforcement Learning,” 2018. URL <https://spinningup.openai.com>.
- [8] Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., and Dormann, N., “Stable-Baselines3: Reliable Reinforcement Learning Implementations,” *Journal of Machine Learning Research*, Vol. 22, No. 268, 2021, pp. 1–8.
- [9] Ng, A. Y., Harada, D., and Russell, S., “Policy invariance under reward transformations: Theory and application to reward shaping,” *ICML*, Vol. 99, 1999, pp. 278–287.
- [10] Bengio, Y., Louradour, J., Collobert, R., and Weston, J., “Curriculum learning,” *Proceedings of the 26th annual international conference on machine learning*, 2009, pp. 41–48.